

Sprint 13 Structured Notes: Events, Timing, Animations, Canvas, Dialogs, and Event Flow

0. What Sprint 13 Is About

Sprint 13 teaches you how to make a web page respond after something happens.

Before Sprint 13, you learned how to:

- store values
- use variables
- work with strings, numbers, and booleans
- write functions
- use arrays and objects
- process data with loops and array methods
- select HTML elements from the page
- change page text, attributes, classes, styles, and child elements

Sprint 13 adds the next major skill:

Make the page react to users and time-based changes.

A real web page does not only run code once. It waits for things to happen.

Examples:

```
A user clicks a button.  
A user types in an input.  
A user submits a form.  
A timer finishes.  
An animation frame is ready.  
A dialog opens.  
A child element inside a list is clicked.
```

Sprint 13 teaches how JavaScript listens for those moments and runs code at the right time.

By the end of Sprint 13, you should understand:

- events
- `addEventListener()`
- inline event handlers and why they are not the best default
- the event object
- `event.target`

- `event.currentTarget`
 - `removeEventListener()`
 - `DOMContentLoaded`
 - common events like `click`, `input`, `change`, `keydown`, and `submit`
 - `preventDefault()`
 - `setTimeout()`
 - `setInterval()`
 - `clearInterval()`
 - avoiding duplicate intervals
 - event bubbling
 - `stopPropagation()`
 - event delegation
 - the `<dialog>` element
 - `showModal()`
 - `show()`
 - `close()`
 - `requestAnimationFrame()`
 - Canvas basics
 - `getContext("2d")`
 - simple DOM and event performance habits
 - debounce
 - throttle
-

1. Current Progress Before Sprint 13

Sprint 13 depends on the earlier browser sprints.

Sprint 11 Review

Sprint 11 taught you how JavaScript finds and reads the page.

Example:

```
const title = document.getElementById("title");
console.log(title.textContent);
```

Simple meaning:

Find the HTML element with the id `title`, then read its text.

Sprint 11 was mainly about selecting and reading page elements.

Sprint 12 Review

Sprint 12 taught you how JavaScript changes the page.

Example:

```
const title = document.getElementById("title");  
title.textContent = "New title";
```

Simple meaning:

Find the element, then change its text.

Sprint 12 also taught you safe DOM updates.

Important safety rule:

```
element.textContent = userInput;
```

is safer for user text than:

```
element.innerHTML = userInput;
```

because `textContent` treats the value as plain text, while `innerHTML` treats the value as HTML.

Sprint 13 Main Shift

Sprint 11 was:

Find the page.

Sprint 12 was:

Change the page.

Sprint 13 is:

Change the page when something happens.

Example:

```
<button id="saveBtn">Save</button>
<p id="status">Not saved yet.</p>

<script>
  const button = document.getElementById("saveBtn");
  const status = document.getElementById("status");

  button.addEventListener("click", () => {
    status.textContent = "Saved!";
  });
</script>
```

What happens:

1. JavaScript finds the button.
2. JavaScript finds the paragraph.
3. JavaScript waits for a click.
4. The user clicks the button.
5. The function runs.
6. The paragraph changes.

2. The Core Idea of Events

An event is something that happens on the page.

Examples of events:

```
click
input
change
keydown
submit
DOMContentLoaded
```

Simple meaning:

An event is a signal that something happened.

Examples in real language:

```
The user clicked the button.
The user typed into the input.
```

The form was submitted.
The page finished loading its HTML.
The user pressed a key.

JavaScript can listen for these events and run code when they happen.

3. Event Mental Model

A beginner event flow looks like this:

```
HTML exists
↓
JavaScript selects an element
↓
JavaScript adds an event listener
↓
The user or browser triggers an event
↓
The listener function runs
↓
The page changes
```

Example:

```
<button id="runBtn">Run</button>
<p id="message">Waiting...</p>

<script>
  const button = document.getElementById("runBtn");
  const message = document.getElementById("message");

  button.addEventListener("click", () => {
    message.textContent = "Button clicked.";
  });
</script>
```

Read it as:

When the button is clicked, change the paragraph text.

4. `addEventListener()`

`addEventListener()` lets JavaScript listen for an event.

Basic syntax:

```
element.addEventListener("eventName", functionToRun);
```

Example:

```
const button = document.getElementById("runBtn");

button.addEventListener("click", () => {
  console.log("Button clicked");
});
```

Simple meaning:

Listen for a click on the button. When the click happens, run this function.

5. Parts of `addEventListener()`

Look at this code:

```
button.addEventListener("click", () => {
  console.log("Clicked");
});
```

It has three main parts.

Part 1: The Element

```
button
```

This is the element we are listening to.

Part 2: The Event Name

```
"click"
```

This is the event we are waiting for.

Event names are strings.

Part 3: The Callback Function

```
() => {  
  console.log("Clicked");  
}
```

This function runs later, after the event happens.

This function is also called an event handler or event listener callback.

6. Inline Event Handlers

An inline event handler puts JavaScript directly inside HTML.

Example:

```
<button onclick="alert('Clicked')">Click</button>
```

This works, but it is not the best default for modern beginner code.

Why?

Because it mixes HTML structure with JavaScript behavior.

HTML should describe what is on the page.

JavaScript should describe what happens.

Cleaner version:

```
<button id="btn">Click</button>
```

```
<script>
  const btn = document.getElementById("btn");

  btn.addEventListener("click", () => {
    alert("Clicked");
  });
</script>
```

Beginner rule:

Prefer `addEventListener()` instead of inline event handlers.

7. The Event Object

When an event happens, JavaScript can give your function an event object.

The event object contains information about what happened.

Example:

```
const button = document.getElementById("runBtn");

button.addEventListener("click", event => {
  console.log(event);
});
```

Simple meaning:

The `event` parameter stores details about the click.

The parameter name can technically be different:

```
button.addEventListener("click", e => {
  console.log(e);
});
```

But `event` is clearer for beginners.

8. `event.target`

`event.target` is the element that actually triggered the event.

Example:

```
<button id="saveBtn">Save</button>

<script>
  const button = document.getElementById("saveBtn");

  button.addEventListener("click", event => {
    console.log(event.target.id);
  });
</script>
```

If the user clicks the button, the output is:

```
saveBtn
```

Simple meaning:

`event.target` tells you what element caused the event.

9. `event.currentTarget`

`event.currentTarget` is the element that the event listener is attached to.

Example:

```
<div id="box">
  <button id="innerBtn">Click me</button>
</div>

<script>
  const box = document.getElementById("box");

  box.addEventListener("click", event => {
    console.log("target:", event.target.id);
    console.log("currentTarget:", event.currentTarget.id);
  });
</script>
```

If the user clicks the button, output is:

```
target: innerBtn  
currentTarget: box
```

Why?

Because the click started on the button.

So:

```
event.target
```

is the button.

But the listener was attached to the `div`.

So:

```
event.currentTarget
```

is the `div`.

10. `target` vs `currentTarget`

This distinction matters in event bubbling and event delegation.

Property	Simple Meaning
<code>event.target</code>	The exact element that triggered the event.
<code>event.currentTarget</code>	The element that the listener is attached to.

Example sentence:

I clicked a button inside a list. The button is the target. The list may be the `currentTarget` if the listener is on the list.

11. `removeEventListener()`

`removeEventListener()` removes an event listener.

Basic syntax:

```
element.removeEventListener("eventName", functionReference);
```

Important rule:

You must pass the same function reference that was used with `addEventListener()`.

Correct:

```
const button = document.getElementById("runBtn");

function handleClick() {
  console.log("Clicked");
}

button.addEventListener("click", handleClick);

// Later:
button.removeEventListener("click", handleClick);
```

This works because `handleClick` is the same function reference in both places.

12. Common `removeEventListener()` Mistake

This does not work correctly:

```
button.addEventListener("click", () => {
  console.log("Clicked");
});

button.removeEventListener("click", () => {
  console.log("Clicked");
});
```

The two arrow functions look the same, but they are not the same function object.

Simple meaning:

JavaScript sees them as two different functions.

So the second function does not remove the first function.

Beginner rule:

Use a named function when you may need to remove the listener later.

13. DOMContentLoaded

Sometimes JavaScript runs before the HTML element exists.

Problem example:

```
<script>
  const button = document.getElementById("runBtn");

  button.addEventListener("click", () => {
    console.log("Clicked");
  });
</script>

<button id="runBtn">Run</button>
```

This can fail because JavaScript searches for the button before the browser creates it.

If `document.getElementById("runBtn")` cannot find the element, it returns:

```
null
```

Then this fails:

```
button.addEventListener("click", ...)
```

because `null` does not have `addEventListener()`.

14. Fixing Script Timing Problems

Solution 1: Put the Script at the End of `<body>`

```
<button id="runBtn">Run</button>

<script>
```

```
const button = document.getElementById("runBtn");

button.addEventListener("click", () => {
  console.log("Clicked");
});
</script>
```

This works because the button exists before JavaScript tries to select it.

Solution 2: Use `DOMContentLoaded`

```
document.addEventListener("DOMContentLoaded", () => {
  const button = document.getElementById("runBtn");

  button.addEventListener("click", () => {
    console.log("Clicked");
  });
});
```

Simple meaning:

Wait until the HTML has been parsed, then run this setup code.

Beginner rule:

If your script is in the `<head>` or before the HTML elements, use `DOMContentLoaded` or move your script to the end of `<body>`.

15. Common User Events

Sprint 13 introduces these common events:

```
"click"

☐
☐


```

Each event is useful for a different situation.

16. The `click` Event

The `click` event fires when the user clicks an element.

Example:

```
<button id="btn">Click me</button>
<p id="message">Waiting...</p>

<script>
  const btn = document.getElementById("btn");
  const message = document.getElementById("message");

  btn.addEventListener("click", () => {
    message.textContent = "Clicked.";
  });
</script>
```

Use `click` for:

- buttons
- menus
- delete actions
- open actions
- close actions
- toggles

17. The `input` Event

The `input` event fires as the user types or changes input content.

Example:

```
<input id="nameInput" />
<p id="preview"></p>

<script>
  const input = document.getElementById("nameInput");
  const preview = document.getElementById("preview");

  input.addEventListener("input", () => {
    preview.textContent = input.value;
  });
</script>
```

```
});  
</script>
```

What happens:

1. The user types into the input.
2. The `input` event fires.
3. JavaScript reads `input.value`.
4. JavaScript updates the paragraph.

Simple meaning:

Use `input` when you want to respond while the user is typing.

18. The `change` Event

The `change` event usually fires after the user commits a value.

Example:

```
input.addEventListener("change", () => {  
  console.log("Value committed:", input.value);  
});
```

Simple meaning:

Use `change` when you care about the final selected or committed value.

Examples where `change` is common:

- select dropdowns
 - checkboxes
 - radio buttons
 - inputs after the user leaves the field
-

19. `input` vs `change`

Event	When It Fires	Good Use
<code>input</code>	As the value changes	Live preview, live search, live character counter
<code>change</code>	After the value is committed	Dropdown selection, checkbox change, final form value

Example:

```
input.addEventListener("input", () => {  
  console.log("Typing now:", input.value);  
});  
  
input.addEventListener("change", () => {  
  console.log("Final value:", input.value);  
});
```

20. The `keydown` Event

The `keydown` event fires when the user presses a key.

Example:

```
input.addEventListener("keydown", event => {  
  console.log(event.key);  
});
```

If the user presses Enter, `event.key` is:

```
Enter
```

Example use:

```
input.addEventListener("keydown", event => {  
  if (event.key === "Enter") {  
    console.log("Enter was pressed");  
  }  
});
```

Use `keydown` for:

- keyboard shortcuts
 - detecting Enter
 - detecting Escape
 - custom keyboard interactions
-

21. The `submit` Event

The `submit` event fires when a form is submitted.

Example:

```
<form id="loginForm">
  <input id="username" />
  <button>Submit</button>
</form>

<script>
  const form = document.getElementById("loginForm");

  form.addEventListener("submit", event => {
    console.log("Form submitted");
  });
</script>
```

Important:

A form can submit when:

- the user clicks a submit button
- the user presses Enter in some form fields
- JavaScript triggers submission

Beginner rule:

Listen for `submit` on the form, not only `click` on the submit button.

22. `preventDefault()`

Some browser behaviors happen automatically.

For example, submitting a form may reload the page or navigate away.

`preventDefault()` stops the browser's default action.

Example:

```
<form id="loginForm">
  <input id="username" />
```

```
<button>Submit</button>
</form>

<script>
  const form = document.getElementById("loginForm");

  form.addEventListener("submit", event => {
    event.preventDefault();
    console.log("Form submit stopped");
  });
</script>
```

Simple meaning:

The form tried to submit normally, but JavaScript stopped the default browser behavior.

Use `preventDefault()` when:

- you want to validate the form first
- you want to handle form data with JavaScript
- you do not want the page to reload
- you are replacing the browser's normal behavior with your own logic

Beginner rule:

Do not call `preventDefault()` randomly. Use it when you intentionally want to stop the default browser action.

23. Reading Input Values

Input elements usually store user text in the `.value` property.

Example:

```
<input id="username" />
<button id="showBtn">Show</button>
<p id="result"></p>

<script>
  const username = document.getElementById("username");
  const showBtn = document.getElementById("showBtn");
  const result = document.getElementById("result");

  showBtn.addEventListener("click", () => {
```

```
    result.textContent = username.value;
  });
</script>
```

Simple meaning:

`username.value` gets what the user typed.

24. Timer APIs

JavaScript can run code later or repeatedly using timer APIs.

The main timer tools in Sprint 13 are:

```
setTimeout()
setInterval()
clearInterval()
```

25. `setTimeout()`

`setTimeout()` runs code once after a delay.

Example:

```
setTimeout(() => {
  console.log("This runs after 2 seconds");
}, 2000);
```

The delay is written in milliseconds.

```
1000 milliseconds = 1 second
2000 milliseconds = 2 seconds
5000 milliseconds = 5 seconds
```

Simple meaning:

Wait for a delay, then run the function once.

26. `setTimeout()` Example with the DOM

```
<p id="status">Waiting...</p>

<script>
  const status = document.getElementById("status");

  setTimeout(() => {
    status.textContent = "Finished.";
  }, 3000);
</script>
```

What happens:

1. The page shows `Waiting...`.
 2. JavaScript waits 3 seconds.
 3. The text changes to `Finished.`
-

27. `setInterval()`

`setInterval()` runs code repeatedly after each delay.

Example:

```
setInterval(() => {
  console.log("This repeats every second");
}, 1000);
```

Simple meaning:

Run this function every 1000 milliseconds.

Use `setInterval()` for:

- timers
 - clocks
 - repeated checks
 - repeated non-visual work
-

28. `clearInterval()`

`clearInterval()` stops an interval.

Example:

```
let count = 0;

const intervalId = setInterval(() => {
  count++;
  console.log(count);

  if (count === 5) {
    clearInterval(intervalId);
  }
}, 1000);
```

Expected output:

```
1
2
3
4
5
```

Then the interval stops.

Simple meaning:

Save the interval ID, then use that ID to stop the interval later.

29. Why Interval IDs Matter

When you create an interval, JavaScript gives you an ID.

Example:

```
const intervalId = setInterval(() => {
  console.log("Running");
}, 1000);
```

The ID lets you stop that specific interval:

```
clearInterval(intervalId);
```

Beginner rule:

If you need to stop an interval later, store its ID.

30. Common Timer Mistake: Duplicate Intervals

Problem:

```
startBtn.addEventListener("click", () => {  
  setInterval(() => {  
    seconds++;  
    renderTime();  
  }, 1000);  
});
```

If the user clicks Start many times, many intervals are created.

Then the timer may count too fast.

Better version:

```
startBtn.addEventListener("click", () => {  
  if (intervalId !== null) {  
    return;  
  }  
  
  intervalId = setInterval(() => {  
    seconds++;  
    renderTime();  
  }, 1000);  
});
```

Simple meaning:

If an interval already exists, do not create another one.

31. Mini-Build: Start, Stop, Reset Timer

This project combines events, DOM updates, intervals, conditions, and functions.

```
<h1>Timer</h1>

<p id="time">0</p>

<button id="startBtn">Start</button>
<button id="stopBtn">Stop</button>
<button id="resetBtn">Reset</button>

<script>
  const time = document.getElementById("time");
  const startBtn = document.getElementById("startBtn");
  const stopBtn = document.getElementById("stopBtn");
  const resetBtn = document.getElementById("resetBtn");

  let seconds = 0;
  let intervalId = null;

  function renderTime() {
    time.textContent = seconds;
  }

  startBtn.addEventListener("click", () => {
    if (intervalId !== null) {
      return;
    }

    intervalId = setInterval(() => {
      seconds++;
      renderTime();
    }, 1000);
  });

  stopBtn.addEventListener("click", () => {
    clearInterval(intervalId);
    intervalId = null;
  });

  resetBtn.addEventListener("click", () => {
    clearInterval(intervalId);
    intervalId = null;
    seconds = 0;
    renderTime();
  });
</script>
```

```
});  
</script>
```

32. Timer Project Breakdown

Selecting Elements

```
const time = document.getElementById("time");  
const startBtn = document.getElementById("startBtn");  
const stopBtn = document.getElementById("stopBtn");  
const resetBtn = document.getElementById("resetBtn");
```

These lines find the HTML elements.

State Variables

```
let seconds = 0;  
let intervalId = null;
```

`seconds` stores the current timer number.

`intervalId` stores the active interval ID.

`null` means:

There is no active interval right now.

Render Function

```
function renderTime() {  
  time.textContent = seconds;  
}
```

Simple meaning:

Put the current `seconds` value on the page.

This is a useful pattern.

Keep the DOM update inside a function.

Start Button

```
startBtn.addEventListener("click", () => {  
  if (intervalId !== null) {  
    return;  
  }  
  
  intervalId = setInterval(() => {  
    seconds++;  
    renderTime();  
  }, 1000);  
});
```

What it does:

1. Waits for the Start button to be clicked.
 2. Checks if a timer is already running.
 3. If one is already running, exits early.
 4. If not, starts a new interval.
 5. Every second, increases `seconds` and updates the page.
-

Stop Button

```
stopBtn.addEventListener("click", () => {  
  clearInterval(intervalId);  
  intervalId = null;  
});
```

What it does:

1. Stops the interval.
2. Sets `intervalId` back to `null`.

This allows Start to work again later.

Reset Button

```
resetBtn.addEventListener("click", () => {  
  clearInterval(intervalId);
```

```
intervalId = null;  
seconds = 0;  
renderTime();  
});
```

What it does:

1. Stops the interval.
2. Clears the interval ID.
3. Resets the timer value to 0.
4. Updates the page.

33. Event Bubbling

Event bubbling means an event can move upward through parent elements.

Example:

```
<div id="parent">  
  <button id="child">Click child</button>  
</div>  
  
<script>  
  const parent = document.getElementById("parent");  
  const child = document.getElementById("child");  
  
  child.addEventListener("click", () => {  
    console.log("child clicked");  
  });  
  
  parent.addEventListener("click", () => {  
    console.log("parent clicked");  
  });  
</script>
```

If the user clicks the button, output may be:

```
child clicked  
parent clicked
```

Why?

Because the click starts on the button and then bubbles up to the parent.

Mental model:

```
button clicked
↓
event bubbles to parent div
↓
parent listener also runs
```

34. Why Bubbling Matters

Bubbling matters because a listener on a parent can hear events from child elements.

This can be useful.

It allows event delegation.

But it can also surprise beginners if a parent listener runs when they only expected the child listener to run.

Beginner rule:

If a parent listener runs unexpectedly, check whether event bubbling is happening.

35. `stopPropagation()`

`stopPropagation()` stops an event from continuing upward.

Example:

```
child.addEventListener("click", event => {
  event.stopPropagation();
  console.log("child clicked");
});
```

Now clicking the child does not trigger the parent's click listener.

Simple meaning:

Stop this event from bubbling farther.

Use this carefully.

If you use `stopPropagation()` too much, your event behavior can become harder to understand.

Beginner rule:

Prefer understanding bubbling before stopping it.

36. Event Delegation

Event delegation means:

Put one event listener on a parent element, then use `event.target` to figure out which child was clicked.

This is useful when there are many similar child elements.

Bad pattern:

```
button1.addEventListener("click", handleClick);
button2.addEventListener("click", handleClick);
button3.addEventListener("click", handleClick);
```

Better pattern:

```
parent.addEventListener("click", event => {
  // inspect event.target
});
```

Simple meaning:

Instead of giving every child its own listener, give the parent one listener.

37. Event Delegation Example

```
<ul id="tasks">
  <li><button data-id="1">Delete task 1</button></li>
  <li><button data-id="2">Delete task 2</button></li>
  <li><button data-id="3">Delete task 3</button></li>
</ul>

<script>
```

```
const tasks = document.getElementById("tasks");

tasks.addEventListener("click", event => {
  if (event.target.tagName !== "BUTTON") {
    return;
  }

  console.log("Delete task:", event.target.dataset.id);
});
</script>
```

What happens:

1. The listener is attached to the `<ul>`.
2. The user clicks a button inside the `<ul>`.
3. The click bubbles up to the `<ul>`.
4. The listener runs.
5. `event.target` tells which button was clicked.
6. JavaScript reads the button's `data-id`.

38. Why Event Delegation Is Useful

Event delegation is useful because:

- it reduces the number of event listeners
- it works well for long lists
- it works well for elements added later
- it keeps code cleaner
- it improves performance in event-heavy pages

Example situation:

You have 100 task buttons.

Weak approach:

```
100 buttons = 100 listeners
```

Delegation approach:

```
1 parent list = 1 listener
```

Beginner rule:

Use event delegation when many child elements should behave similarly.

39. Checking the Clicked Element in Delegation

In delegation, always check what was clicked.

Example:

```
tasks.addEventListener("click", event => {  
  if (event.target.tagName !== "BUTTON") {  
    return;  
  }  
  
  console.log("Button clicked");  
});
```

Why?

Because the user might click the `<ul>` or `<li>`, not only the button.

The guard clause protects your code.

This part:

```
if (event.target.tagName !== "BUTTON") {  
  return;  
}
```

means:

If the clicked element is not a button, stop this function.

40. `dataset` and `data-*` Attributes

HTML can store custom data using `data-*` attributes.

Example:

```
<button data-id="3">Delete task 3</button>
```

JavaScript can read it with `dataset` :

```
console.log(event.target.dataset.id);
```

Output:

```
3
```

Simple meaning:

`data-id="3"` becomes `dataset.id` in JavaScript.

This is useful when buttons represent specific items.

41. Dialogs

HTML has a built-in `<dialog>` element.

A dialog is a popup-like box controlled by JavaScript.

Example:

```
<button id="openBtn">Open Dialog</button>

<dialog id="modal">
  <p>This is a dialog.</p>
  <button id="closeBtn">Close</button>
</dialog>

<script>
  const openBtn = document.getElementById("openBtn");
  const closeBtn = document.getElementById("closeBtn");
  const modal = document.getElementById("modal");

  openBtn.addEventListener("click", () => {
    modal.showModal();
  });

  closeBtn.addEventListener("click", () => {
    modal.close();
  });
</script>
```

```
});  
</script>
```

What happens:

1. The user clicks Open Dialog.
2. JavaScript calls `modal.showModal()`.
3. The dialog opens.
4. The user clicks Close.
5. JavaScript calls `modal.close()`.
6. The dialog closes.

42. Dialog Methods

The main dialog methods are:

```
dialog.showModal();  
dialog.show();  
dialog.close();
```

`showModal()`

```
dialog.showModal();
```

Opens the dialog as a modal.

Simple meaning:

The dialog becomes the main active thing on the page.

`show()`

```
dialog.show();
```

Opens the dialog as non-modal.

Simple meaning:

The dialog opens, but it does not behave as strongly like a modal overlay.

`close()`

```
dialog.close();
```

Closes the dialog.

43. `requestAnimationFrame()`

`requestAnimationFrame()` is used for smooth visual animation.

Basic example:

```
function animate() {  
  console.log("Draw next frame");  
  requestAnimationFrame(animate);  
}  
  
requestAnimationFrame(animate);
```

Simple meaning:

Ask the browser to run this function before the next screen repaint.

The browser controls the timing so animation can be smoother.

44. `requestAnimationFrame()` vs `setInterval()`

Tool	Best For
<code>setInterval()</code>	Timers, clocks, repeated non-visual tasks
<code>requestAnimationFrame()</code>	Smooth visual animations

Use `setInterval()` for a simple stopwatch.

Use `requestAnimationFrame()` for movement on screen.

Beginner rule:

For animation loops, prefer `requestAnimationFrame()`.

45. Simple Animation Example

```
<div id="box" style="width: 50px; height: 50px; background: gray; position:
relative;"></div>

<script>
  const box = document.getElementById("box");
  let x = 0;

  function animate() {
    x++;
    box.style.left = `${x}px`;

    if (x < 200) {
      requestAnimationFrame(animate);
    }
  }

  requestAnimationFrame(animate);
</script>
```

What happens:

1. `x` starts at 0.
2. The animation function increases `x`.
3. The box moves to the right.
4. If `x` is less than 200, another frame is requested.
5. The animation stops at 200.

46. Canvas Basics

A `<canvas>` is a drawing surface on a web page.

Example HTML:

```
<canvas id="board" width="300" height="150"></canvas>
```

JavaScript can draw on it.

Example:

```
<canvas id="board" width="300" height="150"></canvas>

<script>
  const canvas = document.getElementById("board");
  const ctx = canvas.getContext("2d");

  ctx.fillRect(20, 20, 100, 50);
  ctx.fillText("Hello", 20, 100);
</script>
```

Simple meaning:

Canvas gives JavaScript a place to draw shapes and text.

47. `getContext("2d")`

To draw on a canvas, you need a drawing context.

```
const canvas = document.getElementById("board");
const ctx = canvas.getContext("2d");
```

`ctx` is the object that gives you drawing methods.

Examples:

```
ctx.fillRect(20, 20, 100, 50);
ctx.fillText("Hello", 20, 100);
```

48. `fillRect()`

`fillRect()` draws a filled rectangle.

Syntax:

```
ctx.fillRect(x, y, width, height);
```

Example:

```
ctx.fillRect(20, 20, 100, 50);
```

Simple meaning:

```
Start 20 pixels from the left.  
Start 20 pixels from the top.  
Draw a rectangle 100 pixels wide.  
Draw it 50 pixels tall.
```

49. fillText()

`fillText()` draws text on the canvas.

Syntax:

```
ctx.fillText(text, x, y);
```

Example:

```
ctx.fillText("Hello", 20, 100);
```

Simple meaning:

Draw the text `Hello` at position $x = 20$ and $y = 100$.

50. Simple DOM and Event Performance Habits

At this level, performance does not mean advanced optimization.

It means avoiding obviously wasteful code.

Good beginner habits:

```
Use event delegation for many similar child elements.  
Avoid duplicate event listeners.  
Clear intervals and timeouts when finished.  
Do not update the DOM more often than needed.
```

Use debounce for search input.
Use throttle for scroll or resize behavior.

51. Avoid Duplicate Event Listeners

Problem:

```
button.addEventListener("click", handleClick);  
button.addEventListener("click", handleClick);  
button.addEventListener("click", handleClick);
```

This may cause the handler to run more than expected.

Better:

```
button.addEventListener("click", handleClick);
```

Beginner rule:

Add a listener once unless you have a specific reason to add more.

52. Clear Intervals When Finished

Problem:

```
setInterval(() => {  
  console.log("Still running");  
}, 1000);
```

This interval keeps running forever unless stopped.

Better:

```
const intervalId = setInterval(() => {  
  console.log("Running");  
}, 1000);  
  
clearInterval(intervalId);
```

Beginner rule:

If repeated work should stop, keep the interval ID and clear it later.

53. Avoid Unnecessary DOM Updates

Changing the DOM too often can make code slower and harder to reason about.

Less efficient pattern:

```
for (const item of items) {  
  list.textContent += item;  
}
```

Better beginner pattern:

```
let text = "";  
  
for (const item of items) {  
  text += item;  
}  
  
list.textContent = text;
```

Simple meaning:

Prepare the result first, then update the DOM once.

54. Debounce

Debounce means:

Wait until events stop happening for a short time, then run the function.

This is useful for search input.

Without debounce, a search function might run on every keypress.

With debounce, it waits until the user pauses typing.

Example debounce function:

```
function debounce(fn, delay) {  
  let timer;  
  
  return function () {  
    clearTimeout(timer);  
    timer = setTimeout(fn, delay);  
  };  
}
```

Use with input:

```
searchInput.addEventListener("input", debounce(() => {  
  console.log("Search now");  
}, 500));
```

Simple meaning:

If the user keeps typing, reset the timer. Run the search only after 500 milliseconds of no new typing.

55. Throttle

Throttle means:

Run a function at most once during a fixed time period.

This is useful for events that fire very often, such as:

- scroll
- resize
- mousemove

Simple example:

```
function throttle(fn, delay) {  
  let waiting = false;  
  
  return function () {  
    if (waiting) {  
      return;  
    }  
  
    fn();  
  };  
}
```

```

    waiting = true;

    setTimeout(() => {
      waiting = false;
    }, delay);
  };
}

```

Use with scroll:

```

window.addEventListener("scroll", throttle(() => {
  console.log("Scroll handled");
}, 500));

```

Simple meaning:

Even if the scroll event fires many times, the function runs at most once every 500 milliseconds.

56. Debounce vs Throttle

Technique	Simple Meaning	Good For
Debounce	Wait until events stop	Search input, live validation
Throttle	Limit how often a function runs	Scroll, resize, mousemove

Beginner memory:

Debounce waits for quiet.
Throttle limits frequency.

57. Minimum Sprint 13 Standard

You understand Sprint 13 when you can explain this code:

```

const list = document.getElementById("tasks");

list.addEventListener("click", event => {
  if (event.target.tagName !== "BUTTON") {
    return;
  }
});

```



```
}  
  
    console.log("Clicked button:", event.target.textContent);  
});
```

58. Minimum Standard Line-by-Line Explanation

Line 1

```
const list = document.getElementById("tasks");
```

Find the parent list element.

Line 2

```
list.addEventListener("click", event => {
```

Listen for clicks on the list.

Lines 3-5

```
    if (event.target.tagName !== "BUTTON") {  
        return;  
    }
```

If the clicked element is not a button, stop the function.

Final Line

```
    console.log("Clicked button:", event.target.textContent);
```

Print the text of the clicked button.

This is event delegation because one parent listener handles clicks from child buttons.

59. Common Beginner Mistakes in Sprint 13

Mistake 1: Selecting Elements Before They Exist

Problem:

```
<script>
  const button = document.getElementById("btn");
  button.addEventListener("click", () => {});
</script>

<button id="btn">Click</button>
```

Fix:

Move the script after the button or use `DOMContentLoaded`.

Mistake 2: Calling the Function Instead of Passing It

Problem:

```
button.addEventListener("click", handleClick());
```

This calls `handleClick()` immediately.

Correct:

```
button.addEventListener("click", handleClick);
```

Pass the function reference without parentheses.

Mistake 3: Removing a Listener with a Different Function

Problem:

```
button.addEventListener("click", () => console.log("Hi"));
button.removeEventListener("click", () => console.log("Hi"));
```

Fix:

```
function sayHi() {  
  console.log("Hi");  
}  
  
button.addEventListener("click", sayHi);  
button.removeEventListener("click", sayHi);
```

Mistake 4: Creating Many Intervals

Problem:

```
startBtn.addEventListener("click", () => {  
  setInterval(runTimer, 1000);  
});
```

Every click creates another interval.

Fix:

```
if (intervalId !== null) {  
  return;  
}
```

Mistake 5: Forgetting `preventDefault()` on Forms

Problem:

```
form.addEventListener("submit", () => {  
  console.log("Submitted");  
});
```

The page may reload before you see the result.

Fix:

```
form.addEventListener("submit", event => {  
  event.preventDefault();
```

```
console.log("Submitted");  
});
```

Mistake 6: Confusing `target` and `currentTarget`

Remember:

```
event.target
```

is what was actually clicked.

```
event.currentTarget
```

is where the listener is attached.

Mistake 7: Using Too Many Listeners Instead of Delegation

Problem:

```
deleteButton1.addEventListener("click", deleteTask);  
deleteButton2.addEventListener("click", deleteTask);  
deleteButton3.addEventListener("click", deleteTask);
```

Better:

```
taskList.addEventListener("click", event => {  
  if (event.target.tagName !== "BUTTON") {  
    return;  
  }  
  
  deleteTask(event.target.dataset.id);  
});
```

60. Practice Plan

Complete these in order.

Practice 1: Button Click

Create a button that changes paragraph text when clicked.

Skills used:

- selecting elements
 - `click`
 - `addEventListener()`
 - `textContent`
-

Practice 2: Input Preview

Create an input that previews typed text in a paragraph.

Skills used:

- `input`
 - `.value`
 - live DOM updates
-

Practice 3: Form Submit

Create a form and stop the page from reloading with `preventDefault()`.

Skills used:

- `submit`
 - event object
 - `preventDefault()`
 - reading input values
-

Practice 4: Timer

Build the start, stop, reset timer.

Skills used:

- `click`
- `setInterval()`
- `clearInterval()`
- state variables
- avoiding duplicate intervals

Practice 5: Event Delegation

Create a list with many child buttons and handle them with one parent listener.

Skills used:

- event bubbling
- `event.target`
- `tagName`
- `dataset`
- delegation

Practice 6: Dialog

Create a dialog that opens and closes.

Skills used:

- `<dialog>`
- `showModal()`
- `close()`
- click events

Practice 7: Timer APIs and Animation

Try one example of each:

```
setTimeout()  
setInterval()  
requestAnimationFrame()
```

Skills used:

- delayed work
 - repeated work
 - animation frame loop
-

61. Revision Checkpoint Questions

Answer these without looking first.

1. What is an event?
2. What does `addEventListener()` do?
3. Why is `addEventListener()` usually better than inline event handlers?
4. What is the event object?
5. What does `event.target` mean?
6. What does `event.currentTarget` mean?
7. What is the difference between `target` and `currentTarget`?
8. Why does `removeEventListener()` need the same function reference?
9. What problem does `DOMContentLoaded` solve?
10. What is the difference between `input` and `change`?
11. What does `keydown` detect?
12. Why should you listen for `submit` on the form instead of only clicking the submit button?
13. What does `preventDefault()` stop?
14. What does `setTimeout()` do?
15. What does `setInterval()` do?
16. What does `clearInterval()` do?
17. Why should you avoid duplicate intervals?
18. What is event bubbling?
19. What does `stopPropagation()` do?
20. What is event delegation?
21. Why does event delegation reduce the number of listeners?
22. What does `dataset` help you read?
23. What does `dialog.showModal()` do?
24. What does `dialog.close()` do?
25. Why is `requestAnimationFrame()` useful for animation?
26. What is a canvas?
27. What does `getContext("2d")` give you?
28. What does `fillRect()` draw?
29. What does `fillText()` draw?
30. What is debounce?
31. What is throttle?

62. Sprint 13 Summary

Sprint 13 teaches you how to make pages respond.

Before Sprint 13, you could find and change HTML elements.

Now you can change them at the right time.

Main idea:

```
Find an element.  
Listen for an event.  
Run a function.  
Update the page.
```

The most important Sprint 13 tools are:

```
addEventListener()  
removeEventListener()  
event.target  
event.currentTarget  
preventDefault()  
setTimeout()  
setInterval()  
clearInterval()  
requestAnimationFrame()
```

The most important Sprint 13 browser concepts are:

```
events  
event object  
DOM loading  
event bubbling  
event delegation  
dialogs  
canvas  
simple performance habits
```

Final beginner memory:

```
Sprint 11: Find the page.  
Sprint 12: Change the page.  
Sprint 13: Change the page when something happens.
```